

Patch 2a Deep Review: State, Registry, Provenance, and the Next Architectural Boundary

Executive summary

The 2a branch is going in the right architectural direction, but I would not yet treat it as ready to become live infrastructure without a small set of correctness fixes. The three core ideas in Claude's Round-5 packet are sound: `state.json` as a deterministic materialized view rather than authority, `registry.yaml` as declarative experiment topology with derived state, and execution receipts emitted at the `run_agent()` boundary rather than optional stage-specific provenance. `filecite` `turnOfile0`

I verified the supplied `concept-research-scout-2a-state.zip` directly. I also attempted to inspect the public `main` and `2a-state` branches through GitHub, but GitHub still returns a cache miss through my web environment, so the ZIP is the primary code evidence for this review. ¹ The ZIP contains no `.git` directory, so I can verify file contents and behavior but **cannot prove the exact commit ancestry or that 9940d38 merged precisely what the packet says it did.**

My local verification produced:

Check	Result
<code>python setup.py</code>	Pass
<code>python -m py_compile scout.py</code> <code>setup.py orchestrator/*.py</code>	Pass
<code>python scout.py doctor</code>	Pass
Targeted charter/result/Git/state/ registry/receipt suite	38/38 pass
State materializer deterministic tests	Pass
Registry structural tests	Pass as written
Rebase-conflict fail-closed test	Pass
Result-interface/path-escape tests	Pass
Charter isolation tests	Pass
Full test suite	Not certified in my environment; cumulative run did not finish inside the execution window, although the relevant targeted tests and isolated troublesome tests pass

Check	Result
Current numbered ideas	44
Current committed <code>state.json</code> files	1
Current committed <code>registry.yaml</code> files	0
Live ledger entities	161
Tombstones visible with <code>include_invalid=True</code>	6
New <code>receipt: 1</code> production receipts	0 ; implementation is presently fixture-tested rather than production-soaked

The most important findings are not architectural repudiations. They are places where the implementation does not yet match its own declared semantics:

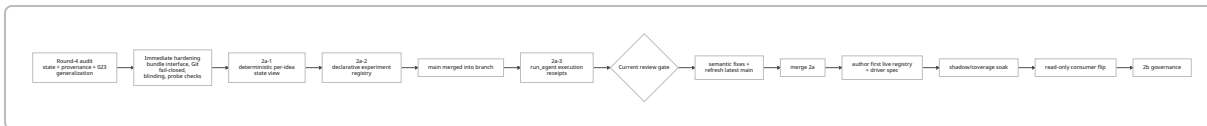
1. `state_view._merged_current()` **does not actually mirror** `ledger.load()` **tombstone semantics.**
2. **The registry currently allows a node to become** `COMPLETE` **despite a declared artifact dependency being absent.**
3. `requires_upstream_bundle` **is launcher metadata but is neither a real DAG dependency nor cycle-checked.**
4. **A contract-hash mismatch before a result bundle exists reports** `UNSTARTED` **, despite the schema saying the node is** `STALE` **.**
5. **Registry-owned** `terminal_statuses` **can redefine** `bundle_complete()` **without being hash-bound to the human approval.**
6. **The new receipt implementation improves coverage but drops** `git_commit` **and CLI-version provenance that the old implementation recorded, and its “requested family” field is normally null for role-driven stages.**
7. `state-verify` **currently gives a reassuring success message while verifying only 1 of 44 ideas;** `registry-validate` **succeeds while validating zero real registries.**
8. **Most urgently for merge mechanics: the 2a ZIP contains an older Idea-023 Colab driver than the more recent** `main (18)` **snapshot.** The newer main snapshot contains the local-SSD/FUSE hardening and extraction-floor fix that is absent from the 2a copy. That creates a concrete integration hazard around the exact experiment you froze.

So my merge-level verdict is:

Approve the architecture; conditionally hold activation of the registry/state consumers until the semantic defects below are fixed.

I would also refresh 2a against the latest `main` *after* the Idea-023 record-result freeze is released, and mechanically verify that the newer 023 driver survives unchanged before merging.

None of this calls for redesigning 2a. Quite the opposite: **finish 2a rather than broadening it.**



Verification and current repository state

The Round-5 packet describes 2a-1 as a view whose defining invariant is “delete → regenerate → byte-identical,” 2a-2 as a declarative DAG with derived statuses, and 2a-3 as universal invocation receipts inside `run_agent()`. Those broad descriptions are accurate.

The targeted tests are genuinely useful rather than trivial happy-path tests. In particular, the branch now tests real Git conflict behavior by creating divergent local and remote commits and verifying that `_push_checkpoint()` aborts the rebase, leaves no rebase state behind, leaves no conflict markers, and does not manufacture a conflict-resolution commit. That is exactly the kind of incident-derived regression test this project benefits from.

Likewise, the state tests establish useful invariants: byte-identical regeneration, isolation from unrelated ledger events, isolation from the global `active_cycle`, stale-approval detection, and detection of manual edits. The registry tests cover ordinary cycle detection, dangling references, forbidden hand-authored status, sibling independence, basic stale propagation, and bundle completion. The result tests exercise the generalized Phase-C interface and manifest path escaping.

The weakness is **coverage of semantic edge cases**, not lack of tests.

Expected versus observed behavior

Invariant	Intended behavior	What I observed	Assessment
<code>state.json</code> is reproducible	Delete/regenerate is byte-identical	Targeted tests pass	PASS
Another idea cannot alter this idea's state	Idea-local event scope	Test passes	PASS
Global active cycle cannot alter state	Target identity beats mutable process state	Test passes	PASS
State merge mirrors <code>ledger.load()</code>	Same latest-wins/tombstone semantics	Counterexample below diverges	FAIL
<code>state-verify</code> represents portfolio coverage	Current state corpus is verified	1 state file for 44 ideas; 43 silently skipped	INCOMPLETE

Invariant	Intended behavior	What I observed	Assessment
Registry contains no hand-set status	Status purely derived	Validator rejects <code>status</code> , <code>state</code> , <code>complete</code>	PASS
A declared artifact dependency is binding	Missing/drifted required input prevents valid completion	Missing artifact can coexist with <code>COMPLETE</code>	FAIL
Upstream-bundle requirement is part of DAG semantics	Cannot run before upstream; cycles impossible	Not included in status blocking or cycle graph	FAIL
Contract pin mismatch means stale registry node	Mismatch → <code>STALE</code>	Before bundle exists → <code>UNSTARTED</code>	FAIL
Independent siblings remain independent	Unrelated failed branch does not block sibling	Test passes	PASS
Registry validation represents live coverage	Live DAGs verified	No real <code>registry.yaml</code> exists	NOT SOAKED
Bundle requirements come from contract	Phase C no longer forced through Idea-004 interface	Targeted tests pass	PASS
Bundle cannot escape through chunk manifest	Relative paths constrained to bundle	Test passes	PASS
Contract-declared required output paths cannot escape	Same trust boundary should apply	Not currently enforced	FAIL
Malformed contract fails closed	Contract is authority	<code>required_outputs</code> parser catches exceptions and falls back to legacy mode	FAIL-OPEN
Every agent invocation gets a receipt	Receipt written at execution primitive	Fixture tests pass	PASS structurally
Receipt records requested vs effective family	Rotation visible	Ordinary role-driven calls record requested family as <code>null</code>	PARTIAL
New receipts preserve previous provenance	At least git/tool provenance retained	<code>git_commit</code> + CLI versions disappeared	REGRESSION

Invariant	Intended behavior	What I observed	Assessment
Rebase conflict fails closed	No rescue through conflicted tree	Targeted test + workflow inspection pass	PASS
Workflows no longer use <code>pull --rebase ... true</code>	No swallowed rebase	None remain; only <code>rebase --abort true</code> cleanup remains	PASS
Both test runners are a CI gate	Packet/operating constraint	<code>check.yml</code> invokes only the unittest script	PARTIAL

There is another encouraging piece of current-state evidence: **backpressure has actually landed**. The configured cap is 20 viable `SCOUT_ONLY` candidates. Using the repository's own `_viable_backlog()` semantics, the current snapshot has 64 viable baseline candidates and 16 ISLES candidates. That means automatic baseline generation is now throttled while ISLES remains below threshold. This is exactly the correction I previously recommended: generation now begins to respond to execution capacity instead of endlessly expanding inventory.

The ledger itself currently resolves to 161 live entities and 167 when tombstones are included. Its live status counts are 80 `SCOUT_ONLY`, 40 `SHORTLISTED`, 30 `REJECTED`, 8 `PAUSED`, and 3 `ACTIVE`; scrutiny is dominated by 120 `SCOUTED`, 40 `DEBATED`, and one `PROBED`. The broad bottleneck therefore remains downstream scientific execution, not idea supply.

State, ledger, and registry integrity

The per-idea state architecture is right

I strongly agree with the core decision to make:

```
ideas/NNN/state.json
```

a **materialized view**, not something agents or humans edit as authority.

That is much better than replacing one mutable global JSON file with 44 mutable per-idea JSON files.

The correct conceptual flow is:



That separation should remain the governing model.

The tombstone parity covenant is currently false

`orchestrator/state_view.py:61-74` says `_merged_current()` mirrors `ledger.load()`. It doesn't.

Current state code does:

```

tombstoned = False
for record in lines:
    if record["status"] == "INVALID_ROW":
        tombstoned = True
        merge(record)

return {} if tombstoned else merged

```

Current `ledger.load()` instead merges every event and excludes the entity only when the **final merged status** is `INVALID_ROW`.

I reproduced this sequence:

```
ACTIVE
→ INVALID_ROW
→ ACTIVE
```

`ledger.load()` returns an active entity.

`state_view._merged_current()` returns `{}`.

The current six repair tombstones do not exercise that sequence, so the test passes while the semantic covenant is broken.

I would use the minimal parity fix now:

```
def _merged_current(lines):
    merged = {}
    for ln in lines:
        rec = json.loads(ln)
        for k, v in rec.items():
            if v not in (None, ""):
                merged[k] = v
    return {} if merged.get("status") == "INVALID_ROW" else merged
```

Longer term you may decide that `INVALID_ROW` should be **terminal** and impossible to resurrect. I can see an argument for that. But then change `ledger.load()` and the append rules intentionally in the same patch. Do not have the two materializers silently implement different philosophies.

The state watermark is not a complete state watermark

There is a subtler issue in `state_view.py:11-16`.

The documentation says the source-event watermark carries recency and implies:

same watermark → same bytes.

But `source_event_watermark` hashes only the idea's ledger lines.

The materialized state also depends on:

- `idea_card.json` through charter resolution;
- `probe_contract.yaml`;
- `HUMAN_APPROVED_PROBE`;
- `registry.yaml`;
- arbitrary filenames included in `artifact_files`.

The branch's own stale-approval test demonstrates the problem: mutate the contract without touching the ledger and the state bytes change while the ledger watermark remains exactly the same.

The core deterministic-regeneration invariant is still valid. The **watermark semantics** are what are wrong.

I recommend a complete source manifest:

```
{
  "materialization": {
    "materializer_version": 1,
    "sources": {
      "ledger_events_sha256": "...",
      "idea_card_sha256": "...",
      "contract_blob": "...",
      "approval_sha256": "...",
      "registry_sha256": "..."
    },
    "source_fingerprint_sha256": "..."
  }
}
```

Then the combined fingerprint actually means:

these exact authoritative inputs determine these exact state bytes.

Two state fields should not yet say `[]`

I would remove or null these for now:

```
"pending_decisions": [],
"corrections": []
```

An empty list semantically says:

the materializer checked the relevant authority and found none.

It hasn't. Those subsystems do not exist yet.

That matters because 2b is specifically supposed to make decision receipts authoritative. Until then:

```
"pending_decisions": null
```

or omitting the field is more honest than asserting an empty set.

Likewise, `artifact_files` is currently only a list of filenames, not artifact identities. An arbitrary extra Markdown file changes the materialized view even though the module describes itself as deriving from “authorities only.” I would either remove this field until execution receipts provide artifact hashes, or eventually replace it with explicit artifact references such as:

```
{
  "path": "interpretation.md",
  "sha256": "...",
  "produced_by_receipt": "..."
}
```

Ledger safety is much improved, but event identity remains future debt

The ledger has made two important improvements:

- malformed JSON now fails loudly;
- `ledger.jsonl` is no longer under `merge=union`.

That is correct.

However, ledger order is still:

physical file order = semantic event order.

`recorded_at` is written, but it is not used to order events. There is no `event_id`, no causal predecessor, and no expected prior revision.

That is acceptable while writes are comparatively sparse and rebase conflicts fail closed. It becomes increasingly risky once 2b adds issue-triggered decisions, an arbiter, or more workflow writers.

Before that automation expansion, I would give each mutating event at least:

```
{
  "event_id": "random-or-UUID",
  "recorded_at": "...",
  "actor": "...",
  "run_id": "...",
  "entity_id": "idea-023",
  "expected_entity_watermark": "..."
}
```

The last field is especially valuable. If two concurrent writers both began from the same entity watermark, a rebase can preserve both append events, but the materializer can recognize a **semantic concurrent update** rather than silently treating whichever line landed last as truth.

That is a much better long-term defense than clever Git merge drivers.

The registry concept is good; the current dependency semantics are not yet safe

This is the main 2a finding.

I constructed a two-node registry:

```
probes:
  - id: A
    results_bundle: results/A
    produces: [foo.csv]
    terminal_statuses: [DONE]

  - id: B
    results_bundle: results/B
    terminal_statuses: [DONE]
    depends_on:
      artifacts:
        - probe: A
          output: foo.csv
          sha256: "0000..."
```

I supplied a terminal `summary.json` for B but no A result and no `foo.csv`.

Observed:

```
registry validation: no errors
A: UNSTARTED
B: COMPLETE
```

That is not a DAG dependency in the scientific sense.

An artifact edge currently becomes relevant only when the file already exists and its hash differs. **Absence is ignored.**

The semantics should be something closer to:

```
consumer has no result:
  missing/unready dependency → BLOCKED
```

```
consumer already has a result:
  dependency disappears or changes → STALE
```

The same problem exists with `requires_upstream_bundle`.

The packet describes this as the launcher's upstream dependency declaration. `filecite0turn0file0`. But the validator's cycle graph only contains `all_of` and artifact edges. I made:

```
A requires bundle B
B requires bundle A
```

and validation accepted it.

I actually recommend **not making** `requires_upstream_bundle` **a second independent dependency system**.

Make the DAG explicit:

```
depends_on:
  all_of:
    - probe: synthetic-calibration

launcher:
  upstream_bundle:
    from_probe: synthetic-calibration
    cli_flag: --phase-s-dir
```

Then validate:

```
launcher.upstream_bundle.from_probe
```

 must already be one of this node's DAG predecessors.

That cleanly separates:

- **why execution depends on another node** → graph;
- **how its bundle is passed to the process** → driver.

Contract staleness also has a real edge-case bug

The packet says pinned contract mismatch means `STALE`. `filecite0turn0file0`

Current `derive_status()` only emits `STALE` when:

```
stale Why and (complete or summary)
```

So a node whose registry pins an old contract but has never run reports:

```
UNSTARTED
```

I reproduced that directly.

I would make a registry pin mismatch `STALE` irrespective of execution history. It means the **node declaration itself no longer describes the current approved experiment**, not merely that old results are stale.

Do not add `any_of` or quorum yet

On Claude's second question: **no, I would not add `any_of`, quorum, voting joins, or generalized boolean DAG expressions yet.**

The first thing that breaks when you move from chains to branches is not lack of quorum. It is already visible:

- launcher addressing is by `phase`, which may become non-unique;
- `results_bundle` uniqueness is not enforced;
- artifact dependencies are incomplete;
- `requires_upstream_bundle` duplicates topology;
- terminal semantics have unclear authority.

Fix those first.

When a real scientific experiment needs “one of these branches is sufficient,” add `any_of` with an actual scientific use case and a preregistered interpretation. “Quorum” in particular sounds much more like orchestration machinery than a scientific experiment primitive.

Results, driver, and execution provenance

Bundle generalization is a real improvement

The Round-4 Idea-004 assumptions are finally being peeled out of the generic result validator.

`validate_bundle()` now takes `required_outputs` from the probe contract when present rather than forcing every experiment to emit:

```
resolved_config.json
environment.txt
```

```
manifest/pair_manifest.csv
M/B phases
```

That is exactly the right direction.

The targeted Phase-C tests pass, and chunk-manifest paths are constrained to the bundle using resolved-path containment.

But two fail-closed defects remain.

Contract parsing silently falls back to legacy behavior

`_contract_required_outputs()` catches every exception and returns:

```
[]
```

which means:

malformed new-style contract → silently behave like an old Idea-004 contract.

That is the wrong failure direction for an authority file.

A malformed contract must produce:

contract invalid; bundle cannot be verified.

Likewise, malformed entries inside `required_outputs` should not simply be filtered out.

`required_outputs` **itself needs path containment**

Chunk manifests cannot use:

```
../../outside
```

but a contract's `required_outputs` currently can, because validation performs:

```
(bundle / rel).exists()
```

without the equivalent containment check.

The contract is human-approved, so this is not primarily an attacker scenario. It is an **integrity invariant**: “required output” should mean an artifact inside the result bundle.

Terminal statuses should not ultimately be registry-owned

This is one place where I disagree with the current schema boundary.

Right now:

```
terminal_statuses: [...]
```

lives in `registry.yaml`, and `bundle_complete()` explicitly lets it override the transitional completion literals.

That means editing the registry can change:

whether a scientific experiment is considered complete.

Yet human approval is hash-bound to `probe_contract.yaml`, not to the registry.

That recreates, in a smaller form, the dual-authority problem 2a is trying to eliminate.

My long-term split is:

Probe contract / approved result specification

- scientific phase definition;
- population;
- selection/exclusions;
- metrics;
- required outputs;
- allowed result status vocabulary;
- terminal completion criteria;
- scientific input identity;
- transformations that matter to inference.

Registry

- nodes;
- graph edges;
- node-to-contract/result-spec references;
- result locations;
- driver references;
- derived statuses.

Driver specification

- how approved inputs are staged;
- caching;

- local SSD versus Drive;
- mounting;
- extraction mechanics;
- CLI argument binding;
- checkpoint/resume behavior;
- runtime resource envelope.

Then a registry node can say:

```
result_spec_ref: probe_contract.yaml#/phases/C/result_spec
```

rather than defining its own scientific completion semantics.

A shorter-term alternative would be to include the registry hash in human approval. But even then I prefer scientific terminal semantics in the contract because they are part of the inferential commitment, not merely topology.

`driver_spec.yaml` **is the correct answer to the 023 launcher history**

I agree with Claude's proposed direction strongly.

Idea 023 is now a very good empirical argument for moving driver policy out of Python string templates. Six launcher takes failed for distinct operational reasons before the scientific analysis could proceed, while `run.py`'s scientific gates continued to fail loudly. That distinction is healthy. `filecite` `turn0file0`

I would use approximately:

```
schema_version: 1
idea_id: idea-023
probe_id: phase-c-census

contract_ref: ideas/023/probe_contract.yaml
registry_node: census

staging:
  source:
    kind: zenodo
    record_id: 16813698
    asset_ref: contract.inputs.dataset_archive

cache:
  location: drive

localization:
  policy: local_ssd_required
```

```

extraction:
  include_ref: contract.inputs.required_modalities
  minimum_file_count: 800

upstream_bundles:
  - probe: synthetic-calibration
    cli_flag: --phase-s-dir

arguments:
  data_dir: --data-dir
  output_dir: --output-dir

runtime:
  platform: colab
  resumable: true
  persistent_outputs: drive

verification:
  verify_record_pin: true
  verify_archive_identity: true
  verify_extraction_floor: true

```

There is an important boundary, though:

Do not copy scientific selection rules into `driver_spec.yaml`.

Dataset version, which modalities constitute the scientific input, population inclusion/exclusion, transformations that affect inference, and terminal result semantics belong in the contract. The driver should **reference** them.

The generator should then become genuinely boring:

validated spec in → notebook out.

That is exactly what you want.

The current 2a ZIP creates a concrete 023 merge hazard

This is the most important branch-integration finding.

I compared the supplied 2a snapshot with your more recent `main (18)` snapshot.

The main snapshot's `probes/023/colab_probe_023.ipynb` contains the later local-SSD strategy:

- Drive archive copied to `/content/work`;

- local extraction;
- local file census;
- `>=800` extraction-floor assertion;
- `/content` cwd recovery before recloning.

The 2a snapshot's notebook lacks those changes and still performs heavy extraction through the Drive FUSE tree.

The corresponding `scout.py` diff shows the same thing: the 2a snapshot removes the newer `localize` cell from `_staging_cells()` while adding registry support nearby.

That is precisely the type of branch integration problem the 023 freeze is supposed to prevent.

Because the ZIP contains no Git history, I cannot tell whether Git would automatically preserve main's newer version at merge or produce a conflict. **Do not rely on that.**

The safe order is:

```
023 record-result lands
→ update 2a from the actual latest main
→ assert frozen 023 files exactly match latest main
→ rerun tests
→ merge 2a
→ author 023 registry/driver spec in a fresh patch
```

That is a modification to Claude's proposed merge sequence, and I consider it important.

Receipts belong inside `run_agent()`

This architectural decision is right.

The old approach only wrote provenance at selected `_pipeline_stage()` call sites, meaning debate and specialized loops could fall outside it. Centralizing invocation receipts in `run_agent()` is exactly the correct boundary.

But the new schema is not yet rich enough to become the substrate for 2b.

The old recorder captured:

```
git_commit
claude CLI version
codex CLI version
```

The new recorder does not.

So 2a-3 expands **coverage** while reducing **environment provenance**.

There is also a bug in the claimed requested/effective-family visibility.

Current wrapper:

```
req = agent
...
if eff is None and stage:
    eff = configured_role
...
rec["agent_requested"] = req
```

Normal pipeline calls use:

```
run_agent(..., agent=None, stage="critique")
```

so:

```
"agent_requested": null
```

even though the configured pre-rotation family was known.

What you actually want is:

```
"family_configured": "codex",
"family_effective": "claude"
```

once role freezing exists.

I recommend this eventual execution receipt:

```
{
  "schema_version": 2,
  "receipt_id": "...",
  "run_id": "...",

  "target": {
    "kind": "idea",
    "id": "idea-023",
    "path": "ideas/023",
```

```

    "character": "isles24"
  },
  "stage": "probe_review",
  "role_binding": {
    "binding_id": "...",
    "family_configured": "codex",
    "family_effective": "claude",
    "agents_toml_sha256": "..."
  },
  "prompt": {
    "path": "prompt_probe_review.md",
    "sha256": "..."
  },
  "code": {
    "git_commit": "...",
    "dirty_before": false
  },
  "tools": {
    "claude_cli": "...",
    "codex_cli": "...",
    "python": "..."
  },
  "attempts": [
    {
      "ordinal": 1,
      "model": "...",
      "returncode": 0,
      "duration_s": 42.1,
      "log_sha256": "..."
    }
  ],
  "started_utc": "...",
  "finished_utc": "...",
  "invocation_exit": {
    "class": "ok",
    "detail": null
  }
}

```

I would then create a **separate linked stage-outcome receipt**.

That distinction matters because your current tests intentionally prove:

```
agent execution receipt = ok
subsequent scope guard = failed.
```

That is semantically correct for the agent invocation, but without a stage-outcome record it is easy for future telemetry to interpret:

```
exit_class = ok
```

as:

```
the scientific stage succeeded.
```

Those are not the same thing.

Replace `LAST_RUN` identity with a structured return

Claude explicitly asked whether this larger diff is justified.

Yes.

One concrete failure mode is trivial:

```
a future successful _run_agent_core() code path returns without rebinding LAST_RUN
→ receipt gets classified as skipped.
```

Other problems include nested invocation, stale global state, and the wrapper/core separately computing effective family.

A cleaner contract is:

```
result = _run_agent_core(...)
```

where `result` is something like:

```
AgentRunResult(
    family=...,
    model_requested=...,
    model_used=...,
```

```
    attempts=...,  
    fallback=...,  
    outcome=...,  
)
```

The wrapper serializes it.

One additional detail: `_append_receipt()` currently executes in `finally`. If receipt writing itself throws while an agent exception is already in flight, the provenance-write failure can mask the original exception. Preserve/chains both errors or make receipt failure a clearly differentiated fail-closed exception.

Git, CI, and operational safety

The Git changes are a clear improvement.

`_push_checkpoint()` now:

1. attempts push;
2. rebases if necessary;
3. aborts the rebase on conflict;
4. raises rather than continuing;
5. retries push only after a clean rebase.

That matches Git's own semantics: `git rebase --abort` returns the branch to its pre-rebase state rather than leaving a conflicted worktree in place. ²

I also searched all workflow YAML files. The unsafe:

```
git pull --rebase ... || true
```

pattern is gone.

The remaining:

```
git rebase --abort || true
```

is not the same defect; that is merely best-effort cleanup after the primary rebase has already failed.

The shared `scout-cycle` Actions concurrency group is also useful. GitHub documents that concurrency groups prevent more than one run in a group from running simultaneously, although pending-run handling and ordering are not a transactional queue and should not be treated as one. ³ It also obviously cannot serialize a human Git push against a running Actions job.

The remaining workflow weakness is failed-run checkpoint semantics

All four writer workflows still do something like:

```
git add -A
git commit -m "... workflow checkpoint"

if ! git pull --rebase ...; then
    git rebase --abort || true
    exit 1
fi

git push
```

inside an `always()` step.

So Git conflicts are now safe, but a failed stage can still produce a perfectly clean Git commit containing **partial stage artifacts**.

That can be defensible if those are explicitly forensic checkpoints.

It is less defensible if the commit merely says:

```
workflow checkpoint
```

and ordinary downstream readers can treat its files as successful stage output.

I would eventually make failure persistence explicit:

```
attempts/<run-id>/...
FAILED_RUN.json
stage_outcome = FAILED
```

or only commit paths authorized as failure diagnostics.

The rule should be:

failure evidence may persist; failed scientific artifacts may not masquerade as current stage output.

CI does not yet match the stated two-runner gate

The packet honestly records a runner-parity incident and says both runners are now gated during branch development. `filecite turn0file0`

But `.github/workflows/check.yml` currently executes:

```
pip install
setup.py
py_compile scout.py setup.py orchestrator/ledger.py
doctor
python tests/test_orchestration.py
```

It does **not** run pytest.

It also doesn't compile:

```
orchestrator/state_view.py
orchestrator/experiment_registry.py
```

explicitly, and it doesn't run state/registry integrity commands.

I would harden it to:

```
python -m py_compile scout.py setup.py orchestrator/*.py
python scout.py doctor
python tests/test_orchestration.py
pytest -q tests/test_orchestration.py
```

plus the materialized-state/registry checks once their coverage semantics are fixed.

There is a caveat: I could not get the entire cumulative suite to complete inside this environment's execution window, despite the 38 targeted 2a tests passing and individual suspected tests passing in isolation. I therefore **would not add a blind CI timeout and declare victory**. First determine whether there is an actual order/resource leak or just unusually high test cost.

The current file contains **133 test methods across 34 test classes**. I also AST-checked it for duplicate class names and duplicate method names and found none.

That leads to Claude's merge-concatenation question: a both-inserted test conflict can hide **symbol shadowing**. In Python, a duplicate test method name in one class silently replaces the earlier definition, while duplicate top-level class names can make one class inaccessible under normal name-based discovery.

A cheap static regression is worthwhile:

```
parse tests/*.py with ast
fail on duplicate:
  top-level class names
  top-level function names
  method names within a class
```

Then keep both unittest and pytest collection because the runner-parity incident has already demonstrated that collection/order differences are empirically relevant.

`state-verify` **is not yet a meaningful soak gate**

This is worth emphasizing.

Current repository:

```
44 numbered ideas
1 state.json
0 registry.yaml
```

Yet:

```
python scout.py state-verify
```

prints:

```
state invariant holds: regeneration is byte-identical
```

because missing state files are deliberately skipped.

Similarly:

```
python scout.py registry-validate
```

prints success while no live registry exists.

Those commands are valid rollout tools, but **not evidence that the new architecture has soaked**.

I would add explicit coverage modes:


```
state-verify --existing
state-verify --require-all

registry-validate --existing
registry-validate --require IDEA...
```

and print:

```
verified 1 / 44 materialized idea states
validated 0 registry files
```

even in permissive mode.

A good consumer-flip soak standard is not merely “N nights passed.” I would require:

all numbered ideas materialized; zero old-vs-new shadow mismatches on fields being migrated; seven consecutive scheduled verification runs; and those runs must include real state mutations rather than seven identical snapshots.

At minimum the soak window should observe some combination of:

- a new ledger state transition;
- a contract change and reapproval;
- a registry status transition;
- another idea's events occurring without moving unrelated state.

After that, flip **one read-only consumer first**, such as `status`, before Actioner or dispatch logic.

Future architecture and rulings on Claude's questions

Rulings on the Round-5 questions

Question	Ruling
State schema	Keep state and registry separate. Add source-candidate identity, claim identity/version/hash, role-binding reference, decision-receipt references, latest-result refs/hashes, and a complete source fingerprint. Remove or null placeholder empty arrays until they are genuinely derived.
Registry joins	<code>all_of</code> plus typed artifact edges is enough now. Do not add <code>any_of</code> / quorum preemptively. Fix missing-artifact semantics, launcher dependency duplication, result-location uniqueness, and node addressing first.

Question	Ruling
Terminal statuses	Long-term approved result-spec/contract owned . Registry should reference them. If temporarily registry-owned, approval must bind the registry hash too.
Execution receipts	Good substrate but insufficient for 2b. Add target identity, git commit, config/tool versions, role binding, finish time, logs/artifact hashes. Keep execution, decision, and interrogation records as separate schemas sharing a common envelope.
<code>LAST_RUN</code> identity	Brittle. A successful path that forgets to rebind becomes <code>skipped</code> ; nested/refactored execution can misattribute state. Replace with structured <code>_run_agent_core()</code> result.
Role epoch	Freeze an immutable per-idea resolved role map, not merely a parity integer. Bind it at promotion/first downstream touch and record the <code>AGENTS.toml</code> hash. <code>effective_agent()</code> should consume that binding for numbered ideas and stop consulting global <code>active_cycle</code> .
Consumer soak	Full materialization + shadow comparison + real mutation coverage + roughly seven green scheduled checks. Flip a read-only consumer first. Current 1/44 coverage is nowhere near this threshold.
Both-inserted test merge	Add AST duplicate-symbol detection and keep both runners. Concatenation can silently shadow duplicate methods/classes even when the file parses.
<code>driver_spec.yaml</code>	Yes. Contract owns science; registry owns topology; driver owns staging/runtime/argument plumbing; generator becomes pure renderer. Review and hash-bind the driver package because operational choices can still change what bytes reach the science.
Merge order	Change it to 023 record-result → refresh 2a from latest main → verify zero frozen-023 regression → fix the semantic P0s/run gates → merge 2a → author 023 registry/driver → soak → consumer flip .

Role freezing should store the map, not just “epoch 19”

Today `effective_agent()` still falls back to:

```
load_state()["active_cycle"]
```

when no explicit cycle is supplied.

That rotation confound therefore remains live until the deferred role-binding work lands.

I would create an immutable authority approximately like:

```
{
  "schema_version": 1,
  "binding_id": "idea-023-role-map-v1",
  "idea_id": "idea-023",
  "source_cycle": "isles24-002",
  "agents_toml_sha256": "...",
  "roles": {
    "critique": "claude",
    "revise": "codex",
    "debate_proposer": "claude",
    "debate_critic": "codex",
    "probe_code": "codex",
    "probe_review": "claude",
    "interpret": "claude",
    "interpret_review": "codex"
  }
}
```

I would not put this in `state.json`, because state is a view.

I also would not put it in the scientific kill/idea ledger unless you deliberately want that ledger to become the universal operational event bus.

A small immutable per-idea `role_binding.json`, materialized into `state.json`, is enough.

For legacy ideas, derive from source cycle when provable. Where provenance is genuinely ambiguous, record one explicit migration binding instead of pretending to reconstruct history.

2b should keep decision, interrogation, and execution records distinct

The current receipt work is exactly why I would resist a generic “everything is a receipt” mega-schema.

Use a common envelope:

```
schema_version
record_id
subject artifact path/hash
idea
charter
created_at
actor
references
```

but separate record bodies.

A future `DecisionReceipt` needs:

```
gate/question
artifact_hash
decision = approve | revise | reject
authorized_next_action
resolved_by
source_comment_id
supersedes
```

An `InterrogationRecord` needs:

```
question
subject artifact hash
response_type = EXPLANATION | CHANGE_PROPOSAL
answer/evidence refs
proposed target
base artifact hash
downstream stages invalidated
review required
```

And the execution receipt remains about what process/model ran.

For the interrogation channel, Claude's rule:

answer/propose, never silently edit

is necessary but not sufficient.

The mechanical rule I would impose is:

An `EXPLANATION` has zero state-transition authority. A `CHANGE_PROPOSAL` may name an exact artifact and base hash, but may not modify it; acceptance of the proposal must re-enter every normal gate invalidated by that artifact's change.

That prevents “clarification” from quietly becoming a cheaper revision path.

GitHub's own security guidance is relevant once issue comments become transport: issue and PR bodies, titles, refs, and related context must be treated as untrusted data rather than interpolated into executable shell. ⁴ Restricting the issuing actor and binding `/approve` to an exact artifact hash remain good 2b requirements. GitHub also recommends least-privilege workflow token permissions; the current direction of separating trusted workflow code from result data is consistent with that model. ⁵

Do not accelerate the arbiter because the blackboard architecture is interesting

The blackboard idea is legitimate: recent work has shown a blackboard multi-agent design outperforming several baselines on specific data-discovery tasks. But that evidence is on a narrow task family, not evidence that a research-governance arbiter is automatically beneficial. ⁶

Meanwhile, MAST found 14 recurring multi-agent failure modes spanning system specification, inter-agent alignment, and verification/termination. ⁷

So I remain in favor of the order:

state/receipts/decision semantics first → bounded arbiter later.

When it arrives, **daily action count alone is not the best safety primitive**. I would prefer budgets by category:

```
generation actions/day
literature actions/day
state-mutating actions/day
agent-token/runtime budget
max actions on same idea
max recursive dispatch depth
```

plus:

one idempotency key per proposed action; no rerun on the same subject until its state changes; no agent-created issue automatically triggering another agent-created issue.

GitHub's concurrency primitive helps prevent simultaneous workflow execution, but GitHub explicitly does not promise strong ordering semantics for queued concurrency-group runs, so the blackboard must remain idempotent and state-aware rather than treating the Actions queue as a transaction log. ⁸

The blinding work is evidence-aligned

I approve sibling-score blinding and author-family blinding.

Self-preference in LLM judging is empirically documented, including preferences associated with model familiarity with an output. ⁹ And recent multi-agent-debate work specifically warns that correlated agents can suppress correct minority positions under majority-style aggregation. ¹⁰

That makes your two-family opposition strategy more defensible, but it also argues against treating **agreement itself** as the metric of success.

For telemetry, the single metric I would most want in addition to the planned set is:

reviewer-disagreement and human-override matrix, stratified by proposer family × reviewer family × stage.

For example:

```
Claude proposal → Codex reviewer:
  approve 41%
  revise 44%
  reject 15%
  human overturn reviewer 12%

Codex proposal → Claude reviewer:
  ...
```

That gives you a practical signal for:

- family self-preference;
- reviewer leniency;
- conformity;
- asymmetric opposition;
- whether the human is systematically correcting one pairing.

For reward-hacking risk, your proposed locked/reference recomputation is also well motivated: recent agent evaluation work shows that agents can improve reported metrics by tampering with evaluators or leaking test information, and that evaluator locking plus independent reference computation materially changes this failure mode. ¹¹

Diversity measurement needs another axis, not another agent

The current diversity command is useful but under-sensitive.

Current ISLES output:

```
40 candidates
design_template: 40 tagged
search_mode:    28 tagged
entry_point:    34 tagged
mechanism_family: 0 tagged
```

The top design template is only 25%, so the existing “one template >50%” alarm does not fire.

But the top three templates account for **62.5%** of ISLES candidates.

And among tagged ISLES candidates:

33 of 34 entry points are Entry 2.

Baseline shows a similar top-three design-template concentration of about **59%**, while `mechanism_family` is empty there too.

That means the diversity instrument is already telling you something valuable: **the most concentrated axis isn't necessarily the one with the current alarm.**

I would add, deterministically:

```
missing-tag rate
top-1 share
top-3 share
HHI / effective-number-of-families
```

for each populated axis.

ISLES' design-template HHI is currently about 0.15, corresponding to an effective template count of roughly 6.6, which sounds reasonably diverse until you look at the 97% Entry-2 concentration. That is why multidimensional reporting matters.

Do not add an LLM diversity judge or embeddings yet.

And I still agree with deferring Elo/pairwise tournaments. Before spending $O(n^2)$ evaluation effort, see whether blinding, better source diversity, coarse family quotas, and selective external reseeding materially change the distribution.

Prioritized patch plan and regression tests

I would distinguish **merge/activation correctness** from later architectural improvement.

Priority table

Action	Priority	Recommendation	Gate
A1	P0	Make state-view tombstone semantics exactly match <code>ledger.load()</code> , or deliberately change both together.	Before 2a consumer use; preferably before merge
A2	P0	Fix registry dependency semantics: missing artifact must block/stale; upstream-bundle mapping must correspond to a real DAG edge; cycles must include every true dependency; contract mismatch must stale even pre-run.	Before first live registry

Action	Priority	Recommendation	Gate
A3	P0	Prevent registry from independently weakening scientific completion criteria. Move terminal statuses to approved result spec, or bind registry hash into approval as an interim measure.	Before first live registry
A4	P0	Make contract result-interface parsing fail closed; validate <code>required_outputs</code> type and bundle-relative paths.	Before generalized results become routine
A5	P0	After 023 record-result, refresh 2a against actual latest <code>main</code> and assert the newer local-SSD 023 driver is preserved exactly.	Before branch merge
A6	P0/P1	Harden <code>check.yaml</code> : compile all orchestrator modules, investigate full-suite cumulative runtime/order issue, then make both unittest and pytest true CI gates.	Strongly recommended before merge given the runner-parity incident
A7	P1	Replace <code>LAST_RUN</code> identity with a structured core return; restore git commit/CLI/config provenance; record true configured vs effective family.	Before 2b telemetry relies on receipts
A8	P1	Add linked stage-outcome records after scope/artifact validation so invocation success cannot masquerade as stage success.	Before telemetry/arbitrator
A9	P1	Materialize every idea, add full source fingerprints, and implement shadow comparison before any consumer reads <code>state.json</code> .	Before consumer flip
A10	P1	Freeze immutable per-idea resolved role map + AGENTS config hash; stop numbered-idea stages consulting global <code>active_cycle</code> .	Before more manual/in-flight dispatch
A11	P1	Introduce <code>driver_spec.yaml</code> ; make package-colab a renderer; bind driver provenance to result bundles.	Before more complex multi-phase probes
A12	P1	Add ledger event IDs and an expected prior entity watermark before issue-driven/arbitrator state mutation grows.	Before 2b arbitrator/autodispatch
A13	P1	Make failed workflow checkpoints explicit forensic artifacts rather than ambiguous normal stage outputs.	Before automation broadens
A14	P2	Retire <code>portfolio/ideas.csv</code> writes, <code>_mean_score</code> compatibility naming, generic unsafe scout <code>stage_target()</code> globbing, and old result-layout fallbacks.	2c

Action	Priority	Recommendation	Gate
A15	P2	Turn charters into actual policy bundles—generation policy/rubric/schema—not merely different charter Markdown injected into largely shared interpretability machinery.	After state/registry consolidation
A16	P2	Only then consider arbiter/meta-loop expansion and the self-referential third charter.	Post-2b evidence

Regression tests I would add

Test	Expected invariant
T1 — tombstone/reactivation parity	<code>ACTIVE → INVALID_ROW → ACTIVE</code> yields identical current state in <code>ledger.load()</code> and <code>state_view</code> .
T2 — complete source fingerprint	Contract, approval, registry, or charter-source changes move the combined state source fingerprint even with unchanged ledger events.
T3 — missing artifact dependency	Consumer with no result is <code>BLOCKED</code> ; consumer with an existing result becomes <code>STALE</code> . Never <code>COMPLETE</code> .
T4 — upstream cycle	$A \leftrightarrow B$ through upstream-bundle relationships or their corresponding DAG edges is rejected.
T5 — upstream not complete	Launcher target cannot derive runnable/unblocked state while declared predecessor is incomplete.
T6 — pre-run contract drift	Registry pin mismatch with no bundle yields <code>STALE</code> .
T7 — unique node addressing	Duplicate/ambiguous phase or result-bundle mappings fail unless an explicit shared-bundle construct exists.
T8 — completion authority	Editing unapproved registry metadata alone cannot turn a nonterminal result into complete.
T9 — contract path safety	<code>/tmp/x</code> , <code>../x</code> , and malformed <code>required_outputs</code> fail bundle validation rather than falling back.
T10 — role receipt	On an odd rotation, receipt contains both configured family and effective family, neither null.
T11 — stage failure linking	Invocation <code>ok</code> + scope failure produces execution receipt <code>ok</code> and linked stage outcome <code>FAILED_SCOPE</code> .
T12 — state coverage	<code>state-verify --require-all</code> fails when even one numbered idea lacks state.
T13 — registry coverage	CI reports <code>0 registries validated</code> rather than presenting that as evidence of live-DAG correctness.

Test	Expected invariant
T14 — frozen 023 merge guard	After branch refresh, hashes of frozen <code>probes/023/**</code> match latest <code>main</code> before 2a-specific post-freeze work.
T15 — duplicate Python tests	AST check rejects duplicate top-level test classes and duplicate method names within a test class.
T16 — both runner parity	unittest and pytest collect the expected test count and both return zero.
T17 — failed workflow checkpoint	Failed stage cannot leave an artifact that ordinary stage-completion logic interprets as successful output.

Overall direction

Stepping back from the individual defects, **I like the shape of the system more after 2a than before it.**

The trend is now:

- implicit mutable state → derived state
- bespoke Idea-004 assumptions → declared experiment interfaces
- optional provenance → provenance at the execution boundary
- uncontrolled generation → backlog backpressure
- Git “rescue” behavior → fail-closed synchronization
- opaque human gates → plain-language views with adversarial checking.

That is architectural compression even though the raw codebase has temporarily acquired more files and schemas.

The important thing is to keep the conceptual hierarchy crisp:

```

CHARTER
  defines research objective/policy

IDEA + CLAIM
  defines scientific question

CONTRACT
  defines what empirical claim the experiment may support

REGISTRY
  defines experiment topology

DRIVER SPEC
  defines how approved computation is transported/executed

```

EXECUTION RECEIPT

records what process actually ran

RESULT BUNDLE

records what empirical output was produced

STAGE OUTCOME

records whether that output passed guards/review

DECISION RECEIPT

records what the human authorized

STATE.JSON

summarizes the current state of all of the above
and is authoritative about none of them

If you preserve that separation, 2a is not overengineering. It is the point at which the research scout starts acquiring enough explicit semantics to survive multiple experiments without hard-coding the first one.

The thing I would resist now is **letting the registry, state view, receipts, issue gates, arbiter, and meta-loop all blur into one generalized event framework prematurely**. They serve different epistemic roles. Keep the shared plumbing small while keeping their authority boundaries explicit.

And I would keep the arbiter/meta-loop deferred. The multi-agent literature gives real reasons to investigate blackboard coordination, but it also documents substantial specification, alignment, and verification failures in multi-agent systems; those findings argue for instrumenting your own workflow before increasing its autonomy. ¹² The same is true of debate: cross-family opposition is sensible, but recent evidence that correlated agents can suppress minority-correct answers is another reason to preserve dissent and human adjudication rather than rewarding consensus for its own sake. ¹⁰

What 2a is doing well is **making those future experiments measurable**.

What I am missing

Nothing missing prevents this code review, but three limitations matter:

1. **The public GitHub repository remains inaccessible to my web fetcher**, including the 2a-state branch, so I could not independently compare the ZIP against today's remote branch. ¹
2. **The ZIP has no .git metadata**, so I cannot verify the claimed branch commits or reconstruct whether the stale 023 notebook would cause a merge conflict versus simply being superseded by newer main. I can only establish that the two supplied snapshots differ materially.
3. **The separate "Systems & Literature Survey and Meta-Charter Foundations" document referenced by the prior Round-4 packet is not present in the current snapshot/attachments available to me**. I therefore cannot audit that survey line-by-line. I could still check the relevant future architectural claims against current primary literature, which is what I did above.

The key actionable conclusion is therefore quite narrow:

Do not rethink 2a. Fix the state semantic covenant, fix the registry's actual dependency/authority semantics, harden the receipt contract, and refresh against latest `main` so Idea 023's hard-won driver fixes cannot regress. Then merge, soak the materialized state as a shadow view, and only afterward let anything depend on it.

That is the right path from a clever research prototype toward a dependable research instrument.

1 (no title)

2 <https://git-scm.com/docs/git-rebase>

<https://git-scm.com/docs/git-rebase>

3 <https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency>

<https://docs.github.com/en/actions/concepts/workflows-and-actions/concurrency>

4 <https://docs.github.com/en/actions/concepts/security/script-injections>

<https://docs.github.com/en/actions/concepts/security/script-injections>

5 https://docs.github.com/en/actions/reference/security/securely-using-pull_request_target

https://docs.github.com/en/actions/reference/security/securely-using-pull_request_target

6 12 <https://arxiv.org/abs/2510.01285>

<https://arxiv.org/abs/2510.01285>

7 <https://arxiv.org/abs/2503.13657>

<https://arxiv.org/abs/2503.13657>

8 <https://docs.github.com/en/actions/how-tos/write-workflows/choose-when-workflows-run/control-workflow-concurrency>

<https://docs.github.com/en/actions/how-tos/write-workflows/choose-when-workflows-run/control-workflow-concurrency>

9 <https://arxiv.org/abs/2410.21819>

<https://arxiv.org/abs/2410.21819>

10 <https://arxiv.org/abs/2606.29270>

<https://arxiv.org/abs/2606.29270>

11 <https://arxiv.org/abs/2603.11337>

<https://arxiv.org/abs/2603.11337>